Database Partitioning

Database Partitioning is the technique of dividing a large table into smaller, more manageable pieces called partitions, while keeping everything within a **single database instance**. Each partition stores a subset of the table's data but appears as one logical table to applications.

Core Concept

Instead of one massive table, you split it into multiple smaller physical segments (partitions) that together form one logical table. Queries can target specific partitions instead of scanning the entire table.

Partitioning vs Sharding

Aspect	Partitioning	Sharding
Scope	Single database instance	Multiple database instances
Servers	One database server	Multiple database servers
Complexity	Lower (built into database)	Higher (distributed system)
Scalability	Limited by single server	Nearly unlimited
Use case	Improve performance on large tables	Scale beyond single server capacity

Types of Partitioning

1. **Range Partitioning :** Divide data based on ranges of a column value.

Example - Date-based:

```sql
-- PostgreSQL
CREATE TABLE orders (
    id SERIAL,
    user_id INT,
    amount DECIMAL(10,2),
    created_at DATE
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2021 PARTITION OF orders
    FOR VALUES FROM ('2021-01-01') TO ('2022-01-01');

CREATE TABLE orders_2022 PARTITION OF orders
    FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');

CREATE TABLE orders_2023 PARTITION OF orders
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

CREATE TABLE orders_2024 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

Query behavior:

```sql
-- Only scans orders_2024 partition
SELECT * FROM orders WHERE created_at >= '2024-01-01';

-- Scans orders_2023 and orders_2024 partitions
SELECT * FROM orders WHERE created_at >= '2023-06-01';

-- Scans ALL partitions (no partition pruning)
SELECT * FROM orders WHERE user_id = 12345;
```

Example - Numeric ranges:

```sql
-- MySQL
CREATE TABLE users (
    id INT,
    name VARCHAR(100),
    age INT
)
PARTITION BY RANGE (age) (
    PARTITION p_young VALUES LESS THAN (18),
    PARTITION p_adult VALUES LESS THAN (65),
    PARTITION p_senior VALUES LESS THAN MAXVALUE
);
```

Advantages:

- Easy to understand and implement
- Perfect for time-series data
- Easy to archive old data (drop old partitions)
- Efficient range queries

Disadvantages:

- Can create uneven partitions (hotspots)
- Manual maintenance (create new partitions for new ranges)
- Queries without partition key scan all partitions

Best for:

- Time-series data (logs, orders, events)
- Sequential or numeric keys
- Data with natural ranges (age groups, price brackets)
- When you need to archive old data regularly

2. **List Partitioning :** Partition based on a list of discrete values.

Example - Geographic:

```sql
-- PostgreSQL
CREATE TABLE customers (
    id SERIAL,
    name VARCHAR(100),
    country VARCHAR(50),
    status VARCHAR(20)
) PARTITION BY LIST (country);

CREATE TABLE customers_north_america PARTITION OF customers
    FOR VALUES IN ('USA', 'Canada', 'Mexico');

CREATE TABLE customers_europe PARTITION OF customers
    FOR VALUES IN ('UK', 'France', 'Germany', 'Spain', 'Italy');

CREATE TABLE customers_asia PARTITION OF customers
    FOR VALUES IN ('China', 'Japan', 'India', 'Singapore');

CREATE TABLE customers_other PARTITION OF customers
    DEFAULT;
```

Query behavior:

```sql
-- Only scans customers_europe partition
SELECT * FROM customers WHERE country = 'France';

-- Scans customers_north_america and customers_europe
SELECT * FROM customers WHERE country IN ('USA', 'UK');
```

Advantages:

- Good for categorical data with known values
- Logical grouping of related data
- Can separate frequently accessed data

Disadvantages:

- Need to know all possible values upfront
- Adding new values requires schema changes
- Not suitable for high-cardinality columns

Best for:

- Categorical data (status, country, department)
- Known, limited set of values
- Geographic distribution
- Data segregation by category

3. **Hash Partitioning :** Database applies hash function to partition key, distributes data evenly across partitions.

Example:

```sql
-- PostgreSQL
CREATE TABLE users (
    id SERIAL,
    name VARCHAR(100),
    email VARCHAR(255)
) PARTITION BY HASH (id);

CREATE TABLE users_p0 PARTITION OF users
    FOR VALUES WITH (MODULUS 4, REMAINDER 0);

CREATE TABLE users_p1 PARTITION OF users
    FOR VALUES WITH (MODULUS 4, REMAINDER 1);

CREATE TABLE users_p2 PARTITION OF users
    FOR VALUES WITH (MODULUS 4, REMAINDER 2);

CREATE TABLE users_p3 PARTITION OF users
    FOR VALUES WITH (MODULUS 4, REMAINDER 3);
```

Advantages:

- Even data distribution (no hotspots)
- Automatic balancing
- Good for uniform access patterns
- No manual maintenance

Disadvantages:

- Cannot do efficient range queries
- Adding/removing partitions requires reorganization
- Queries without partition key scan all partitions
- Less intuitive than range/list partitioning

Best for:

- Even distribution is priority
- Single-key lookups (SELECT * FROM users WHERE id = 12345)
- No need for range queries
- High-cardinality partition keys

4. Composite/Multi-Column Partitioning : Partition by multiple columns or combine partitioning types.

Best for:

- Multi-dimensional data (time + geography, time + category)
- Complex query patterns
- Very large datasets needing fine-grained control

Example - Range + List:

```sql
-- PostgreSQL: First by year, then by country
CREATE TABLE sales (
    id SERIAL,
    amount DECIMAL(10,2),
    country VARCHAR(50),
    sale_date DATE
) PARTITION BY RANGE (sale_date);

CREATE TABLE sales_2023 PARTITION OF sales
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01')
    PARTITION BY LIST (country);

CREATE TABLE sales_2023_us PARTITION OF sales_2023
    FOR VALUES IN ('USA');

CREATE TABLE sales_2023_eu PARTITION OF sales_2023
    FOR VALUES IN ('UK', 'France', 'Germany');

CREATE TABLE sales_2023_asia PARTITION OF sales_2023
    FOR VALUES IN ('China', 'Japan', 'India');
```

```
sales (logical table)
├── sales_2023
│   ├── sales_2023_us
│   ├── sales_2023_eu
│   └── sales_2023_asia
└── sales_2024
    ├── sales_2024_us
    ├── sales_2024_eu
    └── sales_2024_asia
```

Advantages:

- Very granular data organization
- Can optimize for multiple query patterns
- Better partition pruning for complex queries

Disadvantages:

- Increased complexity
- More partitions to manage
- Can become over-engineered

5. **Key Partitioning (Primary Key-based)**
   MySQL specific: Uses the table's primary key for partitioning.
   Important: Partition key must be part of primary key or unique index.

```sql
CREATE TABLE orders (
    order_id INT,
    customer_id INT,
    order_date DATE,
    PRIMARY KEY (order_id, order_date)
)
PARTITION BY RANGE (YEAR(order_date)) (
    PARTITION p2021 VALUES LESS THAN (2022),
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025)
);
```

Partition Pruning

The database automatically skips partitions that can't contain relevant data based on the WHERE clause.

Example:

```sql
-- Table partitioned by created_at (monthly)
SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31';

Without pruning: Scans all 48 partitions (4 years monthly)
With pruning: Scans only orders_2024_01 partition

Speed improvement: 48x faster!
```

How it works:

```sql
-- Query execution plan
EXPLAIN SELECT * FROM orders WHERE created_at = '2024-06-15';

Output shows:
Partitions: orders_2024_06 (1 partition scanned instead of all)
```

Effective pruning requires:

- WHERE clause includes partition key
- Partition key comparison allows calculation of partitions needed

Good for pruning:

```sql
WHERE created_at = '2024-01-15'           -- Exact partition
WHERE created_at >= '2024-01-01'          -- Specific partitions
WHERE created_at BETWEEN '2024-01-01' AND '2024-03-31'   -- Range of partitions
```

Poor for pruning:

```sql
WHERE user_id = 12345                     -- Partition key not used
WHERE YEAR(created_at) = 2024             -- Function prevents pruning (use >= and <
WHERE created_at IS NOT NULL              -- Can't determine partitions
```

Benefits of Partitioning

1. **Performance Improvements**
   - Faster Queries
   - Improved indexes: Smaller partitions → smaller indexes → faster index scans and updates
   - Parallel processing: Database can query multiple partitions in parallel
2. **Maintenance Operations**
   - Faster Backups : Backup partitions individually, only backup active partitions frequently
   - Easier Archiving : Drop or detach old partitions instead of DELETE

```sql
-- Slow: DELETE millions of rows
DELETE FROM orders WHERE created_at < '2020-01-01';  -- Hours

-- Fast: Drop partition
DROP TABLE orders_2020;  -- Seconds
```

   - Faster Updates : Index and statistics on smaller partitions rebuild faster
3. **Data Management**
   - Lifecycle Management: Easy to implement data retention policies

```sql
-- Keep 3 years of data, drop older partitions monthly
DROP TABLE IF EXISTS orders_2021_01;
```

   - Organized Data: Logical separation makes data easier to understand and manage
   - Storage Tiering: Put old partitions on slower/cheaper storage, recent on fast SSD
4. **Availability**
   - Partition-level operations: Maintain or rebuild one partition without affecting others
   - Reduced locking: Operations on one partition don't lock entire table

Limitations and Challenges

1. **Query Planning Overhead**
2. **Unique Constraints:** Unique constraints must include partition key
3. **Foreign Keys :**
    a. PostgreSQL: Foreign keys from partitioned table not supported in older versions.
    b. MySQL: Foreign key support varies by version.
4. **Cross-Partition Queries**
5. **Partition Maintenance**

When to Use Partitioning

Use partitioning when:

- Table exceeds 100GB or 100M rows
- Queries consistently filter on same column (date, category)
- Need to archive/purge old data regularly
- Maintenance operations take too long
- Clear partitioning strategy aligns with access patterns

Don't partition when:

- Table is small (<10GB, <10M rows)
- Queries don't include potential partition key
- Most queries need full table scans anyway
- Adding complexity without clear benefit
- Can solve problem with better indexes